Reporte Técnico de Actividades Práctico-Experimentales Nro. 3

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Katheryn Melissa Contento Maldonado
Asignatura	Simulación
Ciclo	5
Unidad	U1: Introducción a la Simulación
Resultado de aprendizaje de la unidad	R1. Aplica análisis estadísticos de los datos de la Simulación para mejorar la calidad de los resultados, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	03
Título de la Práctica	Simulación de un sistema de líneas de espera M/M/1 mediante SimPy
Nombre del Docente	José O. Guamán Q.
Fecha	15 de Octubre de 2026 16 de Octubre 2026
Horario	Jueves 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la Práctica

Simular un sistema de líneas de espera M/M/1 utilizando Python y SimPy, analizando el comportamiento de los clientes, la cola y el servidor.

- Implementar un modelo M/M/1 utilizando la biblioteca SimPy.
- Calcular las principales métricas teóricas del sistema de espera.
- Comparar los resultados teóricos con los obtenidos mediante la simulación

3. Materiales, Reactivos, Equipos y Herramientas

- Python 3.x.
- Biblioteca SimPy.
- NumPy.
- Matplotlib.
- Editor de código: Visual Studio Code, PyCharm o Jupyter Notebook.

4. Equipos y Herramientas

- Computador.
- Procesador de 2 núcleos o superior.
- Mínimo 4 GB de RAM.
- Sistema operativo Linux o macOS

5. Procedimiento / Metodología Ejecutada

La práctica se desarrolló siguiendo los pasos que se describen a continuación. Las cifras de este reporte provienen de ejecuciones reales del programa implementado.

1) Se leyó la guía y se repasaron las características del modelo M/M/1:

2) Se creó un entorno virtual de Python (versión 3.12 o superior) y se instalaron con `pip` las dependencias del proyecto: SimPy 4.1.2, NumPy 2.5.4, Matplotlib 3.11.2 y Streamlit 1.66.0, esta última para la interfaz gráfica. El código se editó en Visual Studio Code.

3) El programa se organizó con el patrón MVC (Modelo–Vista–Controlador) . La responsabilidad de cada archivo se resume en la Tabla 1.

Tabla 1. Archivos del proyecto y su responsabilidad.

Archivo	Responsabilidad
`app.py`	Punto de entrada; conecta la vista con el controlador.
`model/queue_model.py`	Crea el entorno SimPy y el servidor, genera los clientes y registra sus tiempos.
`model/metrics.py`	Calcula las métricas teóricas y valida las tasas ($\lambda < \mu$).
`controller/simulation_controller.py`	Define los escenarios, valida los parámetros, ejecuta el modelo y arma la tabla comparativa.
`view/dashboard.py`	Muestra controles, tarjetas, tablas y gráficas (Streamlit y Matplotlib).
`tests/`	Pruebas automatizadas con `unittest`.

4) Se creó un entorno `simpy.Environment()` y un servidor `simpy.Resource(env, capacity=1)`, que atiende a los clientes en orden de llegada. Para cada cliente se genera un tiempo entre llegadas exponencial con media $1/\lambda$; al llegar, el cliente solicita el servidor y espera hasta obtenerlo. Cuando lo obtiene, se genera su tiempo de servicio exponencial con media $1/\mu$, se lo atiende y se libera el servidor. La parte central del proceso es:

```
with server.request() as request:
    yield request                                   # espera su turno (cola FIFO)
    service_start = env.now
    service_duration = rng.exponential(1.0 / mu_rate)
    yield env.timeout(service_duration)             # el servidor atiende al
cliente
```

De cada cliente se registran la llegada, el inicio del servicio, la espera (inicio del servicio menos llegada), la duración del servicio, la salida y el tiempo total en el sistema. Los números aleatorios se generan con `numpy.random.default_rng(seed)`, de modo que cada ejecución se puede repetir con la misma semilla.

5) Siguiendo el pseudocódigo de la guía, primero se verifica que $\lambda < \mu$; si no se cumple, el programa muestra el mensaje de que el sistema no es estable y no calcula las métricas estacionarias. Si el sistema es estable, se calculan:

$$\rho = \lambda / \mu \qquad L = \lambda / (\mu - \lambda) \qquad L_q = \lambda^2 / [\mu(\mu - \lambda)] \qquad W = 1 / (\mu - \lambda) \qquad W_q = \lambda / [\mu(\mu - \lambda)]$$

La utilización se calcula como λ/μ, de modo que un valor cercano a 1 significa que el servidor está ocupado casi todo el tiempo.

6) Cada ejecución genera una cantidad fija de clientes y avanza hasta que todos terminan su servicio. El intervalo de observación es [0, T], donde T es el instante en que sale el último cliente. A partir de los registros se calculan:

- **W_q simulado:** promedio de los tiempos de espera de todos los clientes.
- **W simulado:** promedio del tiempo entre la llegada y la salida de cada cliente.
- **L_q simulado:** área bajo la curva del número de clientes esperando, dividida entre T. Se actualiza en cada evento real (cuando un cliente empieza a esperar o empieza a ser atendido).
- **Utilización simulada:** tiempo en que el servidor estuvo ocupado dividido entre T.

7) Se ejecutaron los escenarios que pide la guía (Tabla 2), con 10 000 clientes por escenario y la misma semilla (42) para los tres, de modo que la comparación sea justa.

Tabla 2. Escenarios de simulación.

Escenario	λ	μ	Condición	ρ = λ/μ
1	2.0	3.0	Sistema estable	0.6667
2	2.5	3.0	Mayor utilización	0.8333
3	2.9	3.0	Cerca de saturación	0.9667

8) Se ejecutaron las 20 pruebas automatizadas del proyecto con `python -m unittest -v`, que comprueban las fórmulas teóricas, la validación de parámetros, la cantidad de clientes atendidos, la coherencia de los tiempos, que el servidor no atienda a dos clientes a la vez y la reproducibilidad con la misma semilla. Todas se ejecutaron correctamente (OK).

9) Se organizaron los valores teóricos y simulados en tablas, se generaron con Matplotlib las gráficas de la evolución de la cola, los histogramas de los tiempos de espera y las barras comparativas, y se revisó la interfaz local con `streamlit run app.py`. Finalmente se analizó el comportamiento del sistema en cada escenario.

6. Resultados

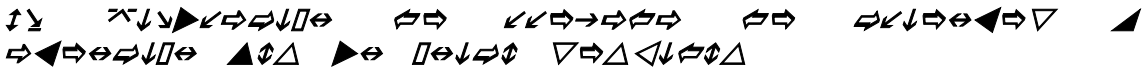
Todos los valores provienen de una ejecución real del programa con 10 000 clientes por escenario y semilla 42.



El modelo quedó implementado en `model/queue_model.py` con un entorno SimPy y un servidor de capacidad 1, y se puede ejecutar desde la interfaz local (Figura 1). Las 20 pruebas automatizadas del proyecto se ejecutaron sin errores (salida: «Ran 20 tests … OK»), lo que respalda el funcionamiento del programa.



Figura 1. Interfaz local del programa comparando los tres escenarios. Esta captura se tomó con la configuración por defecto de la interfaz (1000 clientes, semilla 42), por lo que sus valores simulados difieren de los de las tablas de este reporte (10 000 clientes).



En cada escenario se generaron 10 000 clientes y todos fueron atendidos (Tabla 3). Cada cliente llega, espera si el servidor está ocupado y es atendido en orden de llegada. El programa comprueba internamente que ninguna espera sea negativa, que el tiempo en el sistema sea la suma de la espera y el servicio, y que el servidor nunca atienda a dos clientes al mismo tiempo; si alguna condición falla, la simulación se detiene con un error.

Tabla 3. Clientes generados y atendidos en cada escenario.

Escenario	Clientes generados	Clientes atendidos	Tiempo total simulado T
1	10000	10000	4974.84
2	10000	10000	3964.98
3	10000	10000	3448.57

Como ejemplo, la Tabla 4 muestra el registro de los primeros clientes del escenario 3. Se observa que cada cliente inicia su servicio cuando el anterior ya terminó, y que la espera aparece cuando un cliente llega mientras el servidor está ocupado.

Tabla 4. Registro de los primeros 8 clientes del escenario 3 ($\lambda = 2.9$, $\mu = 3.0$).

Cliente	Llegada	Inicio del servicio	Espera	Servicio	Salida
1	0.8290	0.8290	0.0000	0.7949	1.6240
2	1.6346	1.6346	0.0000	0.0288	1.6634
3	1.7311	1.7311	0.0000	0.4700	2.2011
4	2.2320	2.2320	0.0000	0.0264	2.2584
5	3.3094	3.3094	0.0000	0.0235	3.3328
6	3.6702	3.6702	0.0000	0.5771	4.2474
7	4.0458	4.2474	0.2016	0.0513	4.2986
8	4.1792	4.2986	0.1194	0.0305	4.3291

La Tabla 5 compara el tiempo promedio de espera en la cola simulado con el valor teórico W_q.

Tabla 5. Tiempo promedio de espera en la cola (W_q).

Escenario	W_q teórico	W_q simulado	Diferencia	Diferencia relativa
1	0.6667	0.7087	+0.0420	+6.3 %
2	1.6667	2.0036	+0.3369	+20.2 %
3	9.6667	6.6322	-3.0345	-31.4 %

El tiempo de espera simulado crece de 0.7087 en el escenario 1 a 6.6322 en el escenario 3. También aumenta el máximo individual observado: 7.40, 14.57 y 20.53 unidades de tiempo, respectivamente.

Tabla 6. Tamaño promedio de la cola (L_q).

Escenario	L_q teórico	L_q simulado	Diferencia	Diferencia relativa
1	1.3333	1.4246	+0.0913	+6.8 %
2	4.1667	5.0532	+0.8865	+21.3 %
3	28.0333	19.2317	-8.8017	-31.4 %

La cola promedio simulada pasa de 1.4246 clientes (escenario 1) a 19.2317 clientes (escenario 3), es decir, se multiplica por más de 13 al acercarse λ a μ.

Tabla 7. Utilización del servidor ($\rho = \lambda/\mu$).

Escenario	Cálculo	ρ teórica	ρ simulada	Diferencia
1	2.0 / 3.0	0.6667	0.6696	+0.0030
2	2.5 / 3.0	0.8333	0.8447	+0.0114
3	2.9 / 3.0	0.9667	0.9620	-0.0046

La utilización pasa de 66,7 % a 96,7 %: en el escenario 3 el servidor está ocupado casi todo el tiempo y tiene muy poco margen para absorber la variabilidad de las llegadas. La utilización simulada coincide muy bien con la teórica en los tres escenarios.

Los valores se obtuvieron con las fórmulas de la sección de metodología. Como ejemplo, para el escenario 3 ($\lambda = 2.9$, $\mu = 3.0$):

$$L = 2.9 / (3.0 - 2.9) = 29.0 \qquad L_q = 2.9^2 / [3.0 \times 0.1] = 28.0333$$

$$W = 1 / (3.0 - 2.9) = 10.0 \qquad W_q = 2.9 / [3.0 \times 0.1] = 9.6667$$

Tabla 8. Métricas teóricas del modelo M/M/1.

Escenario	L	L_q	W	W_q	ρ
1	2.0000	1.3333	1.0000	0.6667	0.6667
2	5.0000	4.1667	2.0000	1.6667	0.8333
3	29.0000	28.0333	10.0000	9.6667	0.9667

Los valores son coherentes entre sí: en cada escenario $L = L_q + \rho$ y $L = \lambda W$ (ley de Little). Por ejemplo, en el escenario 3, 28.0333 + 0.9667 = 29.0 y 2.9 × 10.0 = 29.0.

Las Figuras 2 a 4 muestran cómo cambia en el tiempo la cantidad de clientes esperando; la línea discontinua es el valor teórico de L_q. Con $\lambda = 2.0$ la cola es corta y se vacía con frecuencia; con $\lambda = 2.5$ aparecen picos más altos; con $\lambda = 2.9$ la cola alcanza más de 60 clientes y tarda mucho en vaciarse.

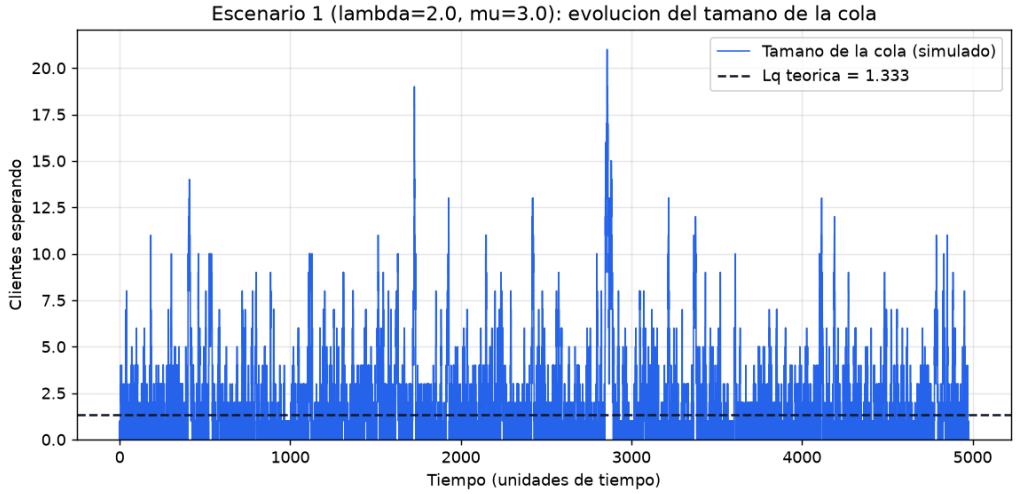


Figura 2. Evolución del tamaño de la cola en el escenario 1 ($\lambda = 2.0$, $\mu = 3.0$).

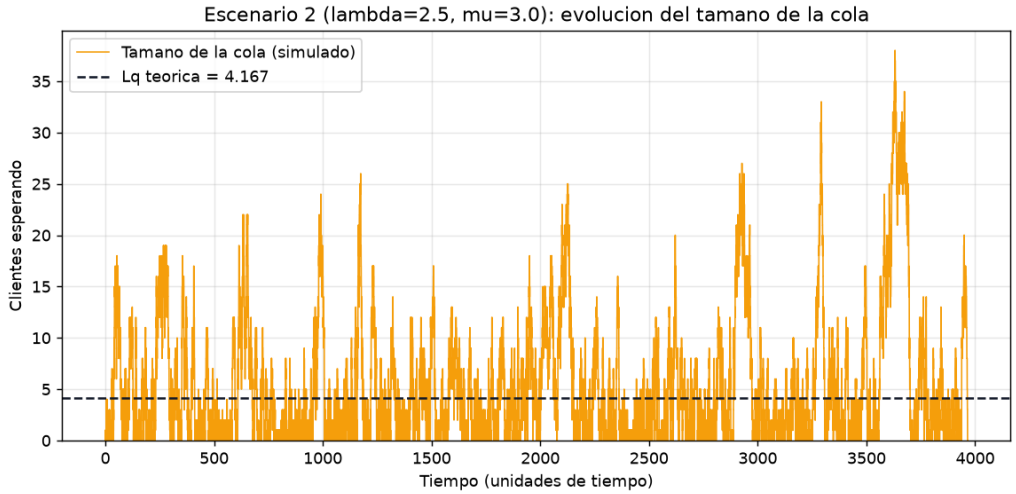


Figura 3. Evolución del tamaño de la cola en el escenario 2 ($\lambda = 2.5$, $\mu = 3.0$).

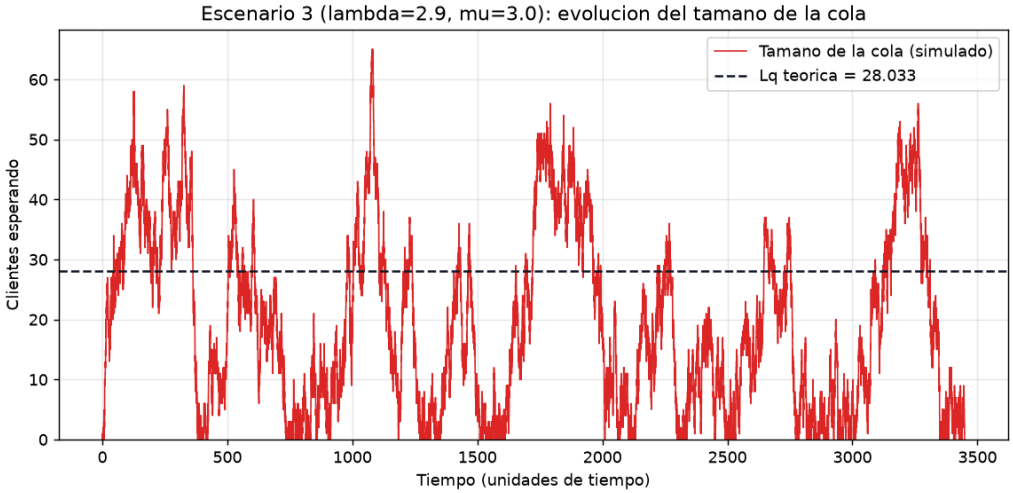


Figura 4. Evolución del tamaño de la cola en el escenario 3 ($\lambda = 2.9$, $\mu = 3.0$).

Las Figuras 5 a 7 muestran la distribución de los tiempos de espera individuales (la línea discontinua es el W_q teórico). El porcentaje de clientes que no esperó nada baja de 33.0 % en el escenario 1 a 16.0 % en el escenario 2 y 3.9 % en el escenario 3, y la distribución se desplaza hacia esperas más largas.

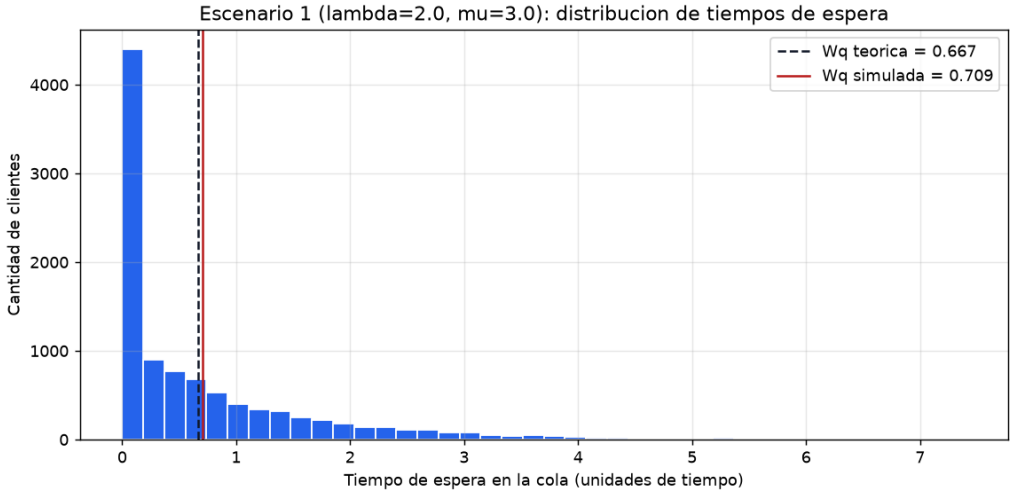


Figura 5. Distribución de los tiempos de espera en el escenario 1.

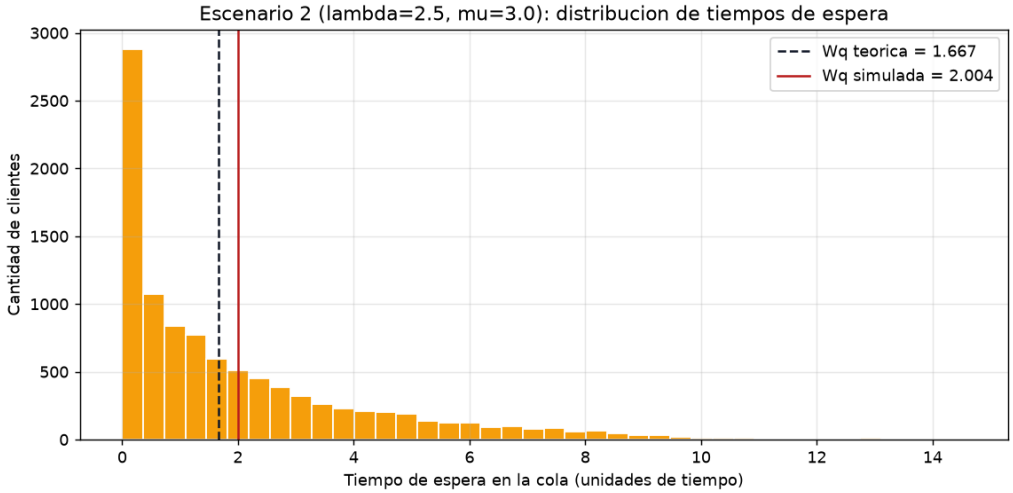


Figura 6. Distribución de los tiempos de espera en el escenario 2.

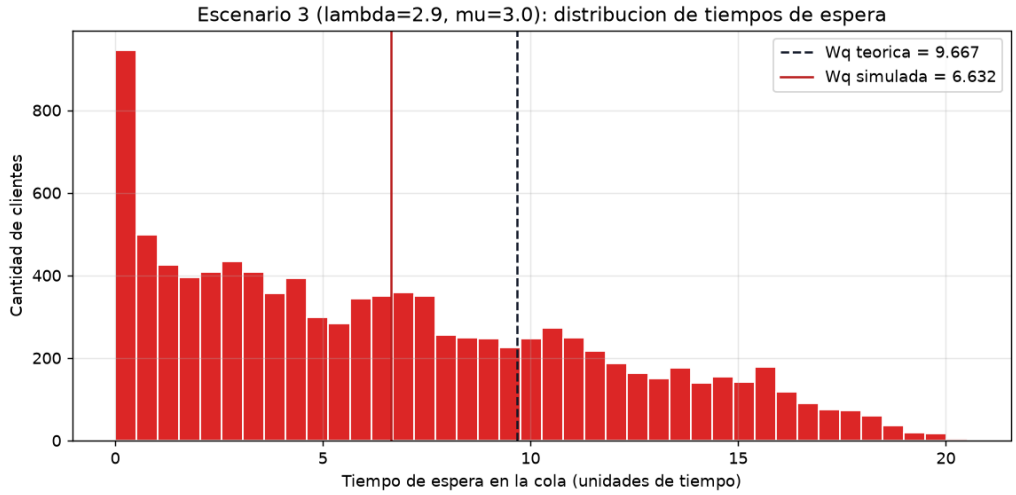


Figura 7. Distribución de los tiempos de espera en el escenario 3.

La Tabla 9 reúne las métricas teóricas y simuladas de los tres escenarios.

Tabla 9. Comparación de métricas teóricas y simuladas (10 000 clientes, semilla 42).

Métrica	Esc. 1 teórico	Esc. 1 simulado	Esc. 2 teórico	Esc. 2 simulado	Esc. 3 teórico	Esc. 3 simulado
ρ	0.6667	0.6696	0.8333	0.8447	0.9667	0.9620
W_q	0.6667	0.7087	1.6667	2.0036	9.6667	6.6322
L_q	1.3333	1.4246	4.1667	5.0532	28.0333	19.2317
W	1.0000	1.0418	2.0000	2.3385	10.0000	6.9639

El programa no calcula L simulado de forma directa; por eso L solo aparece como valor teórico en la Tabla 8.

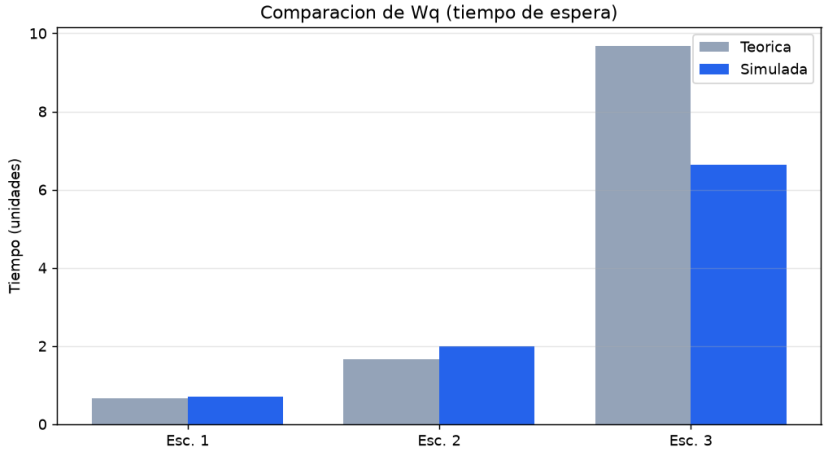


Figura 8. Comparación del tiempo promedio de espera W_q teórico y simulado.

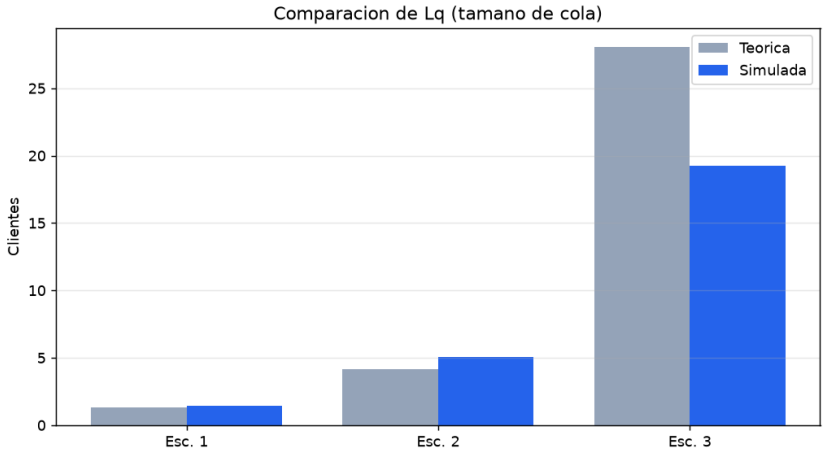


Figura 9. Comparación del tamaño promedio de la cola L_q teórico y simulado.

Escenario 1 (ρ = 0.667, sistema estable): El servidor está libre aproximadamente un tercio del tiempo y el 33 % de los clientes no espera. La cola es corta (L_q simulado 1.4246) y el tiempo de espera simulado (0.7087) queda cerca del teórico (0.6667), con una diferencia de +6.3 %.

Escenario 2 (ρ = 0.833, mayor utilización). Al subir λ de 2.0 a 2.5, la espera simulada se multiplica por casi 3 (2.0036) y la cola pasa a 5.0532 clientes. En esta ejecución el valor simulado quedó por encima del teórico (+20.2 %); como se vio en la comprobación adicional, esto se debe a la variabilidad de una sola réplica, ya que el promedio de 30 réplicas (1.615) queda cerca de la teoría.

Escenario 3 (ρ = 0.967, cerca de la saturación). La espera simulada llega a 6.6322 y la cola a 19.2317 clientes, valores muchos mayores que en los otros escenarios aunque λ solo aumentó de 2.5 a 2.9. Aquí el resultado simulado quedó por debajo del teórico (-31.4 %). Es esperable: cerca de la saturación el sistema es muy sensible al azar (Figura 4 muestra largos periodos de cola alta seguidos de periodos en que se vacía), la simulación arranca con el servidor vacío y la teoría describe el estado estacionario, que necesita simulaciones mucho más largas para alcanzarse. Incluso con 30 réplicas el promedio (8.791) sigue por debajo del valor teórico, lo que confirma que en este escenario la convergencia es lenta.

Se cumple lo esperado en la guía: cuando λ se aproxima a μ, aumentan el tiempo de espera y el tamaño de la cola porque el servidor trabaja cerca de su capacidad máxima. La relación no es lineal: la utilización sube de 0.667 a 0.967 (un 45 %), pero el W_q teórico se multiplica por más de 14. Además, los resultados son consistentes con la ley de Little: por ejemplo, $\lambda \cdot W_q$ simulado = 2.9 × 6.6322 = 19.233, muy próximo al L_q simulado (19.2317), y la utilización simulada (0.6696, 0.8447 y 0.9620) es muy cercana a la teórica.

7. Preguntas de Control

- **¿Qué representa el modelo M/M/1?**

  Representa un sistema de colas con la notación de Kendall M/M/1/∞/∞/FIFO. Las llegadas son de Poisson (primera M), el servicio es exponencial (segunda M) y hay un único servidor. La capacidad y la población son infinitas y la atención es FIFO. Es el caso más simple de la teoría de colas y permite obtener fórmulas cerradas para todas las métricas (diap. 07).

- **¿Qué representan λ y μ?**

  λ es la tasa de llegada: clientes por unidad de tiempo, la demanda del sistema. μ es la tasa de servicio: clientes atendidos por unidad de tiempo, la capacidad del servidor (diap. 15).

- **¿Por qué debe cumplirse λ < μ para que el sistema sea estable?**

Porque equivale a $\rho < 1$: el servidor debe tener más capacidad que la demanda. Si $\lambda \geq \mu$ no existe un estado estacionario con promedios finitos, las colas crecen sin límite y las fórmulas dejan de aplicar (diap. 12 y 21). La ley de Little también exige estabilidad (diap. 10).

- **¿Qué significa la utilización ρ?**
  Es qué tan ocupado está el servidor: $\rho = \lambda/\mu$ (diap. 03 y 12). Por ejemplo, con $\lambda = 8$ y $\mu = 10$, $\rho = 0.8$, es decir, 80 % ocupado y $P_0 = 20$ % libre.
- **¿Cuál es la diferencia entre W y Wq?**
  W es el tiempo esperado total en el sistema (espera + servicio). Wq es el tiempo esperado en cola antes de iniciar el servicio (diap. 09). Se relacionan así: $W = Wq + 1/\mu$.
- **¿Cuál es la diferencia entre L y Lq?**
  L es el número esperado de clientes en el sistema (cola + servicio). Lq es el número esperado de clientes esperando ser atendidos (diap. 09). Se relacionan con W y Wq por la ley de Little: $L = \lambda W$ y $Lq = \lambda Wq$ (diap. 10).
- **¿Por qué SimPy utiliza una distribución exponencial para generar los tiempos?**
  SimPy no la impone: el modelo M/M/1 la exige. Se usa porque describe tiempos aleatorios "sin memoria" (cuánto falta para la próxima llegada no depende de cuánto ya pasó). Eso es lo que hace que las fórmulas teóricas sean exactas.
- **¿Qué sucede con la cola cuando λ se aproxima a μ?**
  Cuando $\rho \rightarrow 1$, todas las métricas tienden a infinito. La causa es la varianza de las distribuciones exponenciales: hay momentos con muchas llegadas y el servidor no puede recuperar el retraso (diap. 13). El crecimiento no es lineal: $L = \rho/(1 - \rho)$ pasa de 4 con $\rho = 0.8$ a 9 con $\rho = 0.9$. En nuestros datos, ρ subió un 45 % (de 0.667 a 0.967), pero el Wq teórico se multiplicó por más de 14. Las diapositivas recomiendan operar con $\rho \leq 0.8$.
- **¿Por qué los resultados simulados pueden ser diferentes de los resultados teóricos?**
  La simulación usa una cantidad finita de números aleatorios, y cada corrida es una muestra distinta.Parte con el sistema vacío, mientras que la teoría describe el estado estacionario.Cerca de la saturación el sistema es muy sensible al azar.
- **¿Qué ocurriría si λ fuera mayor que μ?**
  El sistema sería inestable. No existe estado estacionario con promedios finitos y la cola crece indefinidamente: "¡el sistema colapsa!" (diap. 12). Las fórmulas no aplican (diap. 13). El ejemplo del call center muestra el caso límite: con 5 agentes $\rho = 1.0$ y las colas crecen indefinidamente (diap. 18).

8. Conclusiones

- Se confirmó lo esperado en la guía: al acercarse λ a μ, aumentan el tiempo de espera y el tamaño de la cola. Wq simulado pasó de 0.7087 a 6.6322, y Lq simulado de 1.4246 a 19.2317.
- Las diferencias entre teoría y simulación se deben al azar, al tamaño de la muestra y al arranque con el sistema vacío. Con 30 réplicas los promedios se acercaron a la teoría. En el escenario 3 la convergencia es más lenta.
- Separar el programa en modelo, controlador y vista facilitó probar la simulación sin la interfaz y encontrar errores.

9. Bibliografía / Referencias

Diapositivas semana 3

10. Anexos

Link de GIthub:
https://github.com/Kthiu666/Simulacion_APEs/APE3.git